



Introducción a la base de datos
Trabajo integrador IBD

Grupo nro: 5

Integrante(s):

Matias Fernando Cladera

Spaltro Marcos

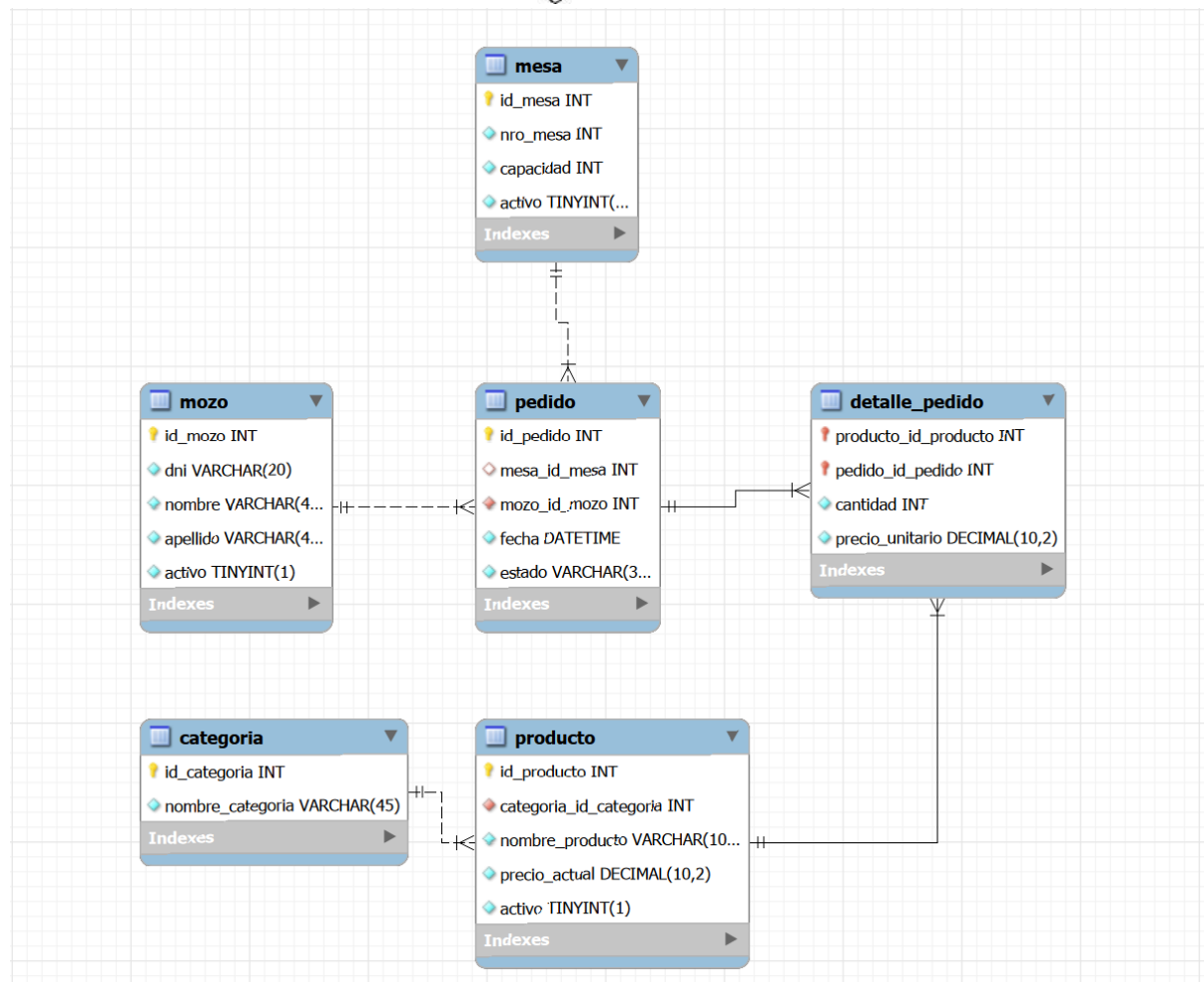
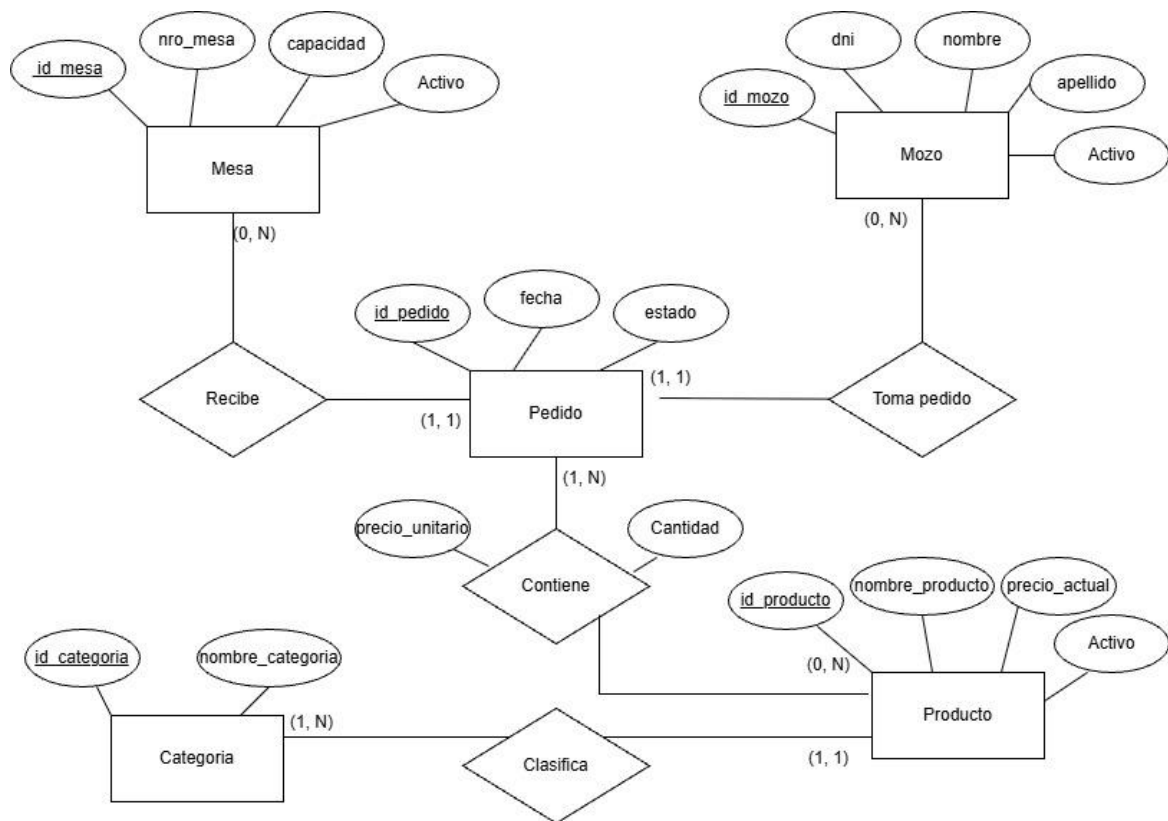
Juan Ignacio Romero

Docente(s):

Lic. Javier Vescio

Lic. Marcelo Wolf

2026



Justificación de decisiones de diseño

objetivo del diseño

El modelo de base de datos fue diseñado con el objetivo de centralizar la operatoria del restaurante, priorizando la escalabilidad, la consistencia histórica de los datos y el cumplimiento de las Formas Normales (hasta 3FN). A continuación, se detallan las decisiones arquitectónicas tomadas para resolver las ambigüedades planteadas en los requerimientos del cliente:

- **Normalización y Clasificación del Menú (Gestión de Categorías)** Frente a la indefinición sobre si los productos comparten características o si se agrupan, se tomó la decisión de crear una entidad fuerte independiente denominada "Categoría". Esto garantiza el cumplimiento de la Tercera Forma Normal (3FN) al eliminar dependencias transitivas y evitar la redundancia de datos en la tabla de productos. Esta estructura permite que el restaurante escale su menú ordenadamente en el futuro.
- **Gestión de Cancelaciones y Modificaciones** El cliente manifestó dudas sobre cómo tratar los pedidos una vez realizados. Para resolver esto sin comprometer la auditoría de la base de datos se implementó el atributo "estado" en la entidad "Pedido". Esto permite gestionar el ciclo de vida de la orden. Asimismo, la utilización de la tabla intermedia "detalle_pedido" permite agregar nuevos productos o actualizar la "Cantidad" de los existentes de manera atómica, respondiendo a la necesidad de gestionar agregados.
- **Consistencia Histórica de Facturación** Para asegurar que los requerimientos evolucionen manteniendo la consistencia de los datos, se incluyó el atributo "precio_unitario" dentro de la tabla intermedia "detalle_pedido". Esta decisión de diseño es fundamental: al copiar el precio del producto en el momento exacto de la venta, se "congela" el valor histórico del ticket. De esta manera, si el restaurante actualiza el "precio_actual" en el catálogo de productos mañana, la facturación de los pedidos cerrados en el pasado no se verá alterada.
- **Flexibilidad en la Asignación de Mesas** Para contemplar "situaciones más complejas" respecto a la asociación exclusiva de pedidos a mesas físicas, se definió que la clave foránea "id_mesa" dentro de la tabla "Pedido" permita valores nulos (NULL). Esta flexibilidad arquitectónica prepara al sistema para registrar operaciones que no ocupan lugar en el salón, tales como pedidos para llevar o consumos directos en la barra.

Funcionamiento de consultas

----- 1) Detalle cuenta

```
SELECT cantidad, nombre_producto, precio_unitario, (cantidad * precio_unitario) AS Total FROM detalle_pedido det
JOIN producto prod ON det.producto_id_producto = prod.id_producto
WHERE det.pedido_id_pedido = 1 ORDER BY id_producto;
```

----- 2) Total cuenta

```
SELECT pedido_id_pedido, SUM(cantidad * precio_unitario) AS Total FROM detalle_pedido WHERE pedido_id_pedido = 1 GROUP BY pedido_id_pedido;
```

- Se deseaba consultar información de pedidos realizados por lo que en la primera consulta se puede consultar el detalle de la cuenta, incluyendo productos, su valor unitario y el valor total de la cuenta o en la segunda consulta ver el total de la cuenta solo sabiendo el id del pedido.

----- 6) Productos

--- Productos más vendidos, con el total *real* ingresado en ventas de este, contemplando la posibilidad de un cambio de precio en el medio

```
SELECT nombre_categoria, prod.nombre_producto, SUM(det.cantidad) AS ventas, SUM(det.cantidad * det.precio_unitario) AS Total
FROM producto prod
JOIN categoria cat ON prod.categoria_id_categoria = cat.id_categoria
LEFT JOIN detalle_pedido det ON prod.id_producto = det.producto_id_producto AND det.pedido_id_pedido IN (SELECT id_pedido FROM pedido WHERE estado = 'completo')
GROUP BY prod.id_producto
ORDER BY ventas DESC, total DESC;
-- Se puede cambiar el ORDER BY por ORDER BY cat.id_categoria, ventas DESC, total DESC; para ordenarlos por categoría y reemplazar POR JOIN el LEFT JOIN para que solo haya productos que tienen ventas
```

--- 7) Venta por categoría

```
SELECT nombre_categoria, SUM(det.cantidad) AS ventas, SUM(det.cantidad * det.precio_unitario) AS Total
FROM producto prod
RIGHT JOIN categoria cat ON prod.categoria_id_categoria = cat.id_categoria
JOIN detalle_pedido det ON prod.id_producto = det.producto_id_producto AND det.pedido_id_pedido IN (SELECT id_pedido FROM pedido WHERE estado = 'completo')
GROUP BY id_categoria
ORDER BY ventas DESC, total DESC;
```

- En cuanto a los productos más solicitados la consulta 6 responde mostrando el producto más consumido, su categoría, el total de ventas y el total ingresado por tales ventas. Para verlo de manera más general también se puede observar por categorías con la consulta 7

----- 3) Mozos

--- Actividad por mozo en rango de fecha específico por el total generado

```
SELECT CONCAT(Mo.Nombre, ' ', Mo.apellido) AS Nombre_completo, COUNT(DISTINCT ped.id_pedido) AS Cantidad_pedidos, SUM(cantidad * precio_unitario) AS Total
FROM pedido ped
JOIN detalle_pedido det ON ped.id_pedido = det.pedido_id_pedido
AND ped.fecha BETWEEN '2026-07-02 00:00:00' AND '2026-07-09 23:59:59'
RIGHT JOIN mozo mo ON ped.mozo_id_mozo = mo.id_mozo
WHERE ped.estado = 'Completo'
GROUP BY mo.id_mozo
ORDER BY Total DESC;
```

--- 4) Actividad por mozo ordenada por el total generado

```
SELECT CONCAT(Mo.Nombre, ' ', Mo.apellido) AS Nombre_completo, COUNT(DISTINCT ped.id_pedido) AS Cantidad_pedidos, SUM(cantidad * precio_unitario) AS Total
FROM pedido ped
JOIN detalle_pedido det ON ped.id_pedido = det.pedido_id_pedido
RIGHT JOIN mozo mo ON ped.mozo_id_mozo = mo.id_mozo
WHERE ped.estado = 'Completo'
GROUP BY mo.id_mozo
ORDER BY Total DESC;
```

- Por el lado de los mozos podremos usar la consulta 3, para ver la actividad de los mozos por un rango específico de tiempo, si quisiésemos ver la de toda la historia podríamos usar por otro lado la consulta 4 que incluye todo el tiempo y además esta ordenado por el total de lo que genero ese mozo, podría cambiar el orden usando ASC en vez de DESC

----- 5) Mesas

--- Actividad por mesa ordenada por el total generado, con el rendimiento según la capacidad de la mesa

--- (El rendimiento ayudaría para evaluar si conviene agregar mesas con mayor o menor capacidad)

```
SELECT me.nro_mesa, COUNT(DISTINCT ped.id_pedido) AS Cantidad_pedidos, SUM(cantidad * precio_unitario) AS Total, ROUND(SUM(cantidad * precio_unitario) / (me.capacidad * COUNT(DISTINCT ped.id_pedido)), 2) AS rendimiento_por_capacidad
FROM pedido ped
JOIN detalle_pedido det ON ped.id_pedido = det.pedido_id_pedido
RIGHT JOIN mesa me ON ped.mesa_id_mesa = me.id_mesa
WHERE ped.estado = 'Completo'
GROUP BY me.id_mesa
ORDER BY Total DESC;
```

- Finalmente, para consultar acerca de las mesas se podrá usar el query 5 en el que se podrá ver las mesas, su capacidad, sus ingresos totales y una tabla de ingresos por personas que servirá para optimizar eficiencia en cuanto a la capacidad de las mesas en el local

Creación de datos

```
--- El cliente debe seleccionar una mesa y un mozo para dar de alta un nuevo pedido, en una mesa disponible (que no tenga pedidos en preparación), mostrados en estos listados:
SELECT * FROM mesa WHERE id_mesa NOT IN (SELECT mesa_id_mesa FROM pedido WHERE estado = 'En preparación') AND activo = true;

SELECT * FROM mozo WHERE activo = true;
```

- Con la primera consulta podremos ver que mesas se encuentran disponibles y con la segunda los mozos

```
--- Para este ejemplo simulamos que el cliente seleccionó la mesa con el id 1 y al mozo con el id 2
INSERT INTO pedido (mesa_id_mesa, mozo_id_mozo, fecha, estado)
VALUES (1, 2, now(), 'En preparación');
```

- Sabiendo los id del mozo y la mesa podremos crear un pedido con nuestros recursos disponibles

```
INSERT INTO categoria (nombre_categoria) VALUES ('Empanadas');
```

- Ingresando un valor entre " se podrá crear una nueva categoría

```
SELECT * FROM categoria;
INSERT INTO producto (categoria_id_categoria, nombre_producto, precio_actual) VALUES (10, 'Helado', 13000);
```

- Se mostrará id y nombre por categoría y se podrá crear un producto ingresando tres valores el id de la categoría a la que pertenece, el nombre del producto y su precio

```
INSERT INTO mozo (dni, nombre, apellido) VALUES (42226897, 'Juan', 'Sanchez');
```

- Sera utilizado para añadir nuevos mozos

```
----- Mesas
--- Aclaración: el campo nro_mesa es para que el cliente pueda numerarlas y reconocerlas a gusto, internamente todas las consultas se van a manejar por el id de la mesa, estás no se pueden eliminar
--- ni modificar su capacidad para evitar incoherencias en los reportes pero si renumerar ya que funciona más como un "nombre", en caso de quererlo que el cliente puede darlas de baja y agregue nuevas mesas para reorganizarlas.

----- Crear
INSERT INTO mesa (nro_mesa, capacidad) VALUES (11, 4);
```

- Para mesas se ha de tener precauciones tales como no poder cambiar la capacidad de las mesas ni eliminarlas porque afectaría otras consultas, por lo que creamos una nueva mesa con la capacidad deseada

Edición

- Primero hemos de saber pedidos en preparación, por lo que usaremos esta consulta.

```
--- Las ediciones se pueden hacer únicamente con pedidos en preparación
--- Una vez dado de alta, al cliente se le mostraría la lista de pedidos para seleccionar, indicando el mozo y la mesa mediante esta consulta:
SELECT id_pedido, mesa_id_mesa, CONCAT(mo.nombre, ' ', mo.apellido) AS mozo FROM pedido ped
JOIN mozo mo ON ped.mozo_id_mozo = mo.id_mozo
WHERE ped.estado = 'En preparación'
ORDER BY mesa_id_mesa;
```

- Podremos ver los productos disponibles y el id de los mismos

```
SELECT prod.id_producto, cat.nombre_categoria, prod.nombre_producto, det.cantidad AS cantidad
FROM producto prod
LEFT JOIN detalle_pedido det ON prod.id_producto = det.producto_id_producto AND det.pedido_id_pedido = 11
JOIN categoria cat ON prod.categoria_id_categoria = cat.id_categoria
WHERE prod.activo = true
GROUP BY prod.id_producto
ORDER BY cantidad DESC, cat.nombre_categoria, prod.nombre_producto;
```

- En el caso de que este producto no haya sido agregado todavía usaremos:

```
--- En caso de que sea un producto aún no agregado: en el ejemplo se usa el prod 1, pedido 11, realizando una subconsulta para agregar el precio
INSERT INTO detalle_pedido (producto_id_producto, pedido_id_pedido, cantidad, precio_unitario)
VALUES (1, 11, 1, (SELECT precio_actual FROM producto WHERE id_producto = 1));
```

- En el caso que sea más de uno podrá actualizar el valor cantidad:

```
UPDATE detalle_pedido SET cantidad = cantidad + 1 WHERE producto_id_producto = 1 AND pedido_id_pedido = 11;
```

- Para restar se usará este caso para cantidad >1:

```
--- En caso de que sean 2 o más y el operador apriete el botón de resta:
UPDATE detalle_pedido SET cantidad = cantidad - 1 WHERE producto_id_producto = 1 AND pedido_id_pedido = 11;
```

- Se mostrarán todas las categorías y su id y se podrá editar el nombre de la categoría usando el id.

```
--- Se muestra por pantalla las categorías para cambiar, una vez hecha la selección se pide el nuevo nombre y se edita
SELECT * FROM categoria;
UPDATE categoria SET nombre_categoria = 'Ensaladas' WHERE id_categoria = 11;
```

- Para cancelar un pedido solo se actualizará su estado sabiendo el id del pedido, no hará falta alterar ninguna otra columna

```
UPDATE Pedido SET estado = 'Cancelado' WHERE id_pedido = 11;
```

- Mostrará los datos de categoría para evitar errores y permitirá actualizar id, nombre y precio

```
SELECT * FROM categoria;
UPDATE producto SET categoria_id_categoria = 9, nombre_producto = 'Affogato', precio_actual = 16000 WHERE id_producto = 40;
```

- Los productos no se pueden eliminar en la tabla producto porque dejaría incongruencias en los pedidos históricos y los ingresos, por lo que se usará una actualización que lo desactive, así mismo se podrá volver a activar cambiando 'false' por 'true'.

```
UPDATE producto SET activo = false WHERE id_producto = 40;
```

- Para la edición de mozos consultaremos los mozos tanto activos como inactivos ya sabiendo el id podremos editar tanto DNI, nombre como apellido o simplemente darlo de baja ajustando el valor de activo a 'false'

```
--- Se muestra un listado completo de mozos, tanto activos como no
```

```
SELECT * FROM mozo;
```

```
--- Una vez seleccionado se procede a editar
```

```
UPDATE mozo SET dni = 123456789, nombre = 'Jhon', apellido = 'Locke' WHERE id_mozo = 6;
```

```
--- Dar de baja o nuevamente de alta (evitando eliminar para no perder redundancia)
```

```
UPDATE mozo SET activo = false WHERE id_mozo = 6;
```

- Se usará la primera consulta para saber los id y la segunda para actualizar el número de la mesa sabiendo su id

```
SELECT * FROM mesa;
```

```
UPDATE mesa SET nro_mesa = 100 WHERE id_mesa = 12;
```

- Para evitar eliminar, que conlleva el mismo problema previamente señalado actualizar las mesas a false es la manera correcta de desactivarlas o "eliminarlas"

```
UPDATE mesa SET activo = false WHERE id_mesa = 12;
```

Eliminación de datos

- En caso de restar 1 a un producto en un pedido cuando la cantidad es 1

```
--- En caso de que sea uno y el operador lo reste:
```

```
DELETE FROM detalle_pedido WHERE producto_id_producto = 1 AND pedido_id_pedido = 11;
```

- Se podrán borrar categorías o mozos introduciendo el id

```
DELETE FROM categoria WHERE id_categoria = 11;
```

```
DELETE FROM mozo WHERE id_mozo = 6 AND id_mozo NOT IN (SELECT mozo_id_mozo FROM pedido);
```

Productos y mozos pueden ser eliminados con precaución (Leer edición de productos), se podrá borrar entonces productos o mozos que NO hayan sido pedidos nunca o que no hayan atendido nunca